Reproducing the results of "Multigrid for the even-odd preconditioned Möbius domain-wall operator with a factorized Pauli–Villars coarse correction"

Wei-Lun Chen

October 27, 2026

This guide accompanies the paper of the title. It describes the implementation in RutticeQCD and how to rerun the measurements of the paper and extend them. Section, table, figure and equation numbers in it refer to the paper; the data and scripts are in `docs/mg_background` of the repository.

I. Implementation and reproduction

Implementation. The method is implemented in RutticeQCD (Rust), on the CPU and on NVIDIA GPUs through CUDA kernels of the Wilson and Möbius operators taken from Bridge++. On the device the fine operators, the prolongation, the coarse operators and the coarse Krylov solvers all stay on the GPU. The precisions are those of Table 3: the prolongation is stored in half precision unless asked otherwise, the cycle above the coarse lattice runs in single precision on operators converted on the device from the links of the double-precision ones, and the outer solver is in double precision. The coarse solvers are BiCGStab for the quark factor and MINRES on $\mathcal{H}_c$ for the Pauli–Villars factor (Sec. III.B), both stopped at $|r|^2/|b|^2 < 10^{-2}$ unless stated, on the even coarse sites. The numbers of Sec. V come from the parameter-file program `ruttice` and from the examples `preconditioned_fov` (CPU) and `preconditioned_fov_device` (GPU), whose options are given with each result; the remarks after Proposition 1 from the example `correction_exactness` (CPU). The numbers of Sec. IV come from the examples `mobius_gamma5_check` (Eqs. (2.10) and (2.13), CPU), `dminus_cost` (the inverse of D_-, GPU), `dual_basis` (prolongations for the dual cycle, GPU) and `dagger_precond` (Table 7, GPU). In HMC the solves with $D^\dagger$ use $\Gamma_5\mathcal{M}\Gamma_5$ (`fermion::adjoint_by_gamma5`). The eigensolvers (thick-restart Lanczos, Krylov–Schur Arnoldi [2] with left eigenvectors, and $\sigma_{\min}(z-X)$ by Lanczos on $(z-X)^\dagger(z-X)$) are in the module `eigen` and are checked against Bridge++'s eigensolver test.

All numbers of Sec. V come from one configuration per volume; the most useful extensions are more configurations, a larger volume and dynamical ensembles. The rest of this appendix gives what is needed to run them.

Code. RutticeQCD, `https://github.com/atsanroger/RutticeQCD`, branch `cuda-bridge` (the commit of each result is given with it; later commits give the same iterations, up to the two-iteration differences between builds noted in Sec. V.B.1). It needs Rust (stable) and, for the GPU, CUDA 12:

```
export CUDA_HOME=/usr/local/cuda
cargo build --release --features cuda-bridge
./target/release/ruttice mg_24.yaml
```

The GPU backend runs on one GPU. The parameter files are in the style of Bridge++ and are documented in `docs/USAGE.md`; gauge configurations are read in the Bridge++ text and binary formats, NERSC, ILDG and SciDAC.

The mass scan (Table 4, Fig. 5). The parameter files are `docs/mg_background/runs/mg_24.yaml` (the multigrid) and `runs/cgne_24.yaml` (the reference, double or mixed precision); the comments in them say how to switch to setup B and to a point source. Each run prints a line

```
seed 91: GCR 28 iterations on the device; 3014 coarse applications;
    worst |Dx-b|^2 2e-18; 13.2 s; ... device 19.91 GB
```

and, for the multigrid, `setup on the device:  206 s`. These, per mass and setup, are what Table 4 lists. On one GPU one configuration takes about 50 minutes for the multigrid (seven masses, both setups, $\alpha = 0.6$; the setup dominates) and under half an hour for the two CGNE scans.

Memory and volume. On $24^3 \times 48$ the multigrid needs 19.9 GB of device memory (CGNE 6.9 GB in double, 7.2 GB mixed), roughly proportional to the volume: $32^3 \times 64$ needs about 63 GB (an 80 GB GPU), $48^3 \times 96$ more than one GPU has. The aggregates, 4^4, must divide the lattice, and each level below the coarse lattice must divide the extent of the level above (`MultiGrid_Level2`, `_Level3`; on $32^3 \times 64$, for instance, super-blocks 2 and 2).

The spectral quantities (Secs. V.D.1–V.D.3). The example `preconditioned_fov_device` computes, with exact coarse solves, the field of values, the parts of the cycle, the eigenvalues with their condition numbers, and $\sigma_{\min}(z - \tilde{A}\mathcal{M})$ on contours, e.g.

```
cargo run --release --features cuda-bridge \
    --example preconditioned_fov_device -- \
    16 16 binary conf.bin 1.0 0.001 1 --ratio 10 \
    --arnoldi --far 1,0 --pseudo --contour 1,0,0.6,16
```

(arguments: L, T, format, file, M_5, m, stout $0/1$). It fits 16^4 in 24 GB and takes about an hour there; $24^3 \times 48$ does not fit. The CPU examples `preconditioned_fov`, `coarse_hermiticity` and `correction_exactness` need no GPU; the last, e.g.

```
cargo run --release --example correction_exactness -- \
    12 binary conf.bin 1.0 0.001 1 --block 4
```

prints $\|e(r) - y\|/\|y\|$ for the right form, the left form and the dual correction, and the GCR(2) iterations of $\tilde{A}_R$ and $\tilde{A}_R^\dagger$ (the smoother of setup S by default, `-steps 12 -ratio 30` for setup B; the exactness checks take about 12 minutes on 12^4 on a desktop CPU).

What would help most.

1. The mass scan of Table 4 on three to five further $24^3 \times 48$ configurations, for the spread of the iterations and of the speed-up.

2. The same on a larger volume ($32^3 \times 64$ on an 80 GB GPU), for the volume dependence of the multigrid iterations and of the setup.

3. The mass scan on a dynamical ($2 + 1$-flavor) ensemble, e.g. that of Ref. [1], where the spectra of Sec. II.A were taken.

4. The spectral run above on further 16^4 configurations, for how the outliers of setup S and the pseudospectra vary between configurations.

For each run, the result lines above and the commit of the build are enough; the logs contain everything else.

References

[1] I. Kanamori, W.-L. Chen and H. Matsufuru, *Spectrum of preconditioned Moebius domain-wall operators*, PoS **LATTICE2024** (2025) 414.

[2] G. W. Stewart, *A Krylov–Schur algorithm for large eigenproblems*, SIAM J. Matrix Anal. Appl. **23** (2001) 601.